

Constella: Lower-Cost Queries over a Fixed Document Index

Dylan Couzon (Qdrant)

October 2026

Abstract

Can we reduce query computation while keeping an existing dense document index? Constella provides three compatible query encoders for Stella, a pretrained 400-million-parameter embedding model: the original encoder, Nano, a 34.5-million-parameter transformer, and Zero, a token lookup table. Across 15 BEIR datasets, Nano and Zero retain 90.5% and 81.4% of Stella’s exact-search score. On a laptop CPU, their median warmed encoding times are 2.25 ms and 44 microseconds, versus Stella’s 31.6 ms.

We study the quality and systems tradeoffs behind these encoding savings. Zero requires more approximate-search effort on the two measured collections; on a million-passage diagnostic, a roughly 62-fold encoding advantage over Nano becomes 2.2-fold after retrieval, at different absolute quality levels. A 26-checkpoint table-fitting study shows why choosing a teacher by its own retrieval score can select a poor student. Earlier experiments show that fusion-operator ordering changes with candidate depth and that checkpoint alignment does not alone certify serving compatibility. Simple routing and vector blending do not deliver the gains their apparent headroom suggests. The contribution is an empirical study of cheaper query paths over fixed document vectors, with findings spanning construction, retrieval, and deployment, and their limits kept visible.

1. Query computation over fixed document vectors

A large embedding model may be affordable when documents are indexed and costly on every query. Switching to an unrelated small model usually means encoding the documents again: the same vector width does not make two models’ coordinates compatible. **Query-side distillation** offers another option. A smaller encoder learns to produce queries in the original model’s document space, allowing the stored document vectors to stay in place.

This paper investigates the quality and retrieval cost of that option. We built **Constella** around **Stella**, the pretrained `stella_en_400M_v5` embedding model [Stella]. Its document vectors are fixed. **Nano** is a small transformer aligned to those vectors. **Zero** looks up and pools learned token vectors, without running a transformer. The original Stella query encoder remains available. An application can choose any of these aligned encoders for a request and search the same collection.

Index reuse and lookup-table queries have precedents. QED, EmbedDistill, LEAF, and NanoVDR study compact query encoders; pyNIFE aligns a static table to a frozen teacher and proposes static/contextual switching [QED, EmbedDistill, LEAF, NanoVDR, pyNIFE]. Model2Vec and static sentence embeddings supply other table-based representations; LightRetriever learns a lookup query side together with its document side [Model2Vec, StaticEmb, LightRetriever]. We claim empirical findings and usable artifacts, not a new architecture or priority for switching.

The useful choices extend beyond encoder size. Which teacher should a cheap encoder imitate? Does the existing approximate-search configuration still work? Would a lexical branch buy more quality than additional query computation? Does the exported model preserve the vectors that were evaluated?

Our results establish four findings:

- **Several query budgets can share the same index.** Nano and Zero offer large encoding savings with measurable relevance loss; keeping Stella’s index is their common constraint.
- **Teacher quality does not rank table quality reliably under the tested recipe.** A direct table screen is more informative than teacher scores or vector agreement.
- **Search work can consume much of the encoding saving.** Additional candidate exploration reduces Zero’s advantage; a controlled query-precision test identifies recoverable candidate information.
- **Encoder quality alone does not determine the deployed result.** Fusion depth changes operator ordering, simple adaptive policies underperform their oracle, and serving wrappers can violate compatibility.

The following sections present the experiments and their implications. “Registered” means the method was fixed before observing the result; “exploratory” identifies later analyses. Detailed methods and the original registered evaluations are in the appendices; the companion evidence package preserves the broader research history.

2. What the three query paths offer

2.1 One compatibility contract

Stella produces normalized 1024-dimensional document vectors. Zero and Nano are trained against that particular space. Zero has 30,522 learned token rows, exported as int8. It pools token vectors with square-root count weights, so repeated tokens gain influence more slowly than simple counting, then normalizes. Nano has 34,540,672 parameters: it starts from bge-small, a pretrained small English embedding model, combines three of its transformer layers, projects their outputs to 1024 dimensions, averages token outputs, and learns to match Stella query vectors using squared L2 loss.

Here, **hot swapping** means selecting an available, aligned encoder without re-encoding documents or building another dense index. We verified all three query paths against the same populated Qdrant collections, with no updates or rebuild between encoder batches. This demonstrates compatibility; model-loading transitions and concurrent failover were not timed. The prompt, tokenizer, pooling, normalization, and model revision remain part of the contract.

2.2 Quality and encoding cost

We use **nDCG@10**, which rewards relevant documents near the top of the first ten results. The BEIR-15 macro average gives equal weight to each of 15 retrieval datasets [BEIR]. Exact search scores every document vector, separating encoder quality from approximate-search errors. **Retention** is the ratio of a student’s macro score to Stella’s, not a percentage of queries answered correctly.

Table 1. Three query paths over identical Stella document vectors. Quality is a descriptive BEIR-15 aggregate. Encoding medians follow a registered protocol: 100 real 5–12-word queries, batch one, four CPU threads, Apple M5 Pro, three fresh processes. Transformers use ONNX Runtime; Zero uses NumPy. Asset sizes include model and tokenizer files.

Query path	Exact nDCG@10	Retention	Encode p50	Assets
Stella	0.5614	100%	31.6 ms	1669.6 MiB
Nano	0.5081	90.5%	2.25 ms	132.3 MiB
Zero	0.4572	81.4%	0.044 ms	90.1 MiB

Nano keeps about nine tenths of Stella’s score at one fourteenth of its encoding time. Zero keeps about four fifths at one seven-hundredth. These are different relevance budgets, not equal-quality speedups. Zero’s warmed encoding is small, but tokenization, asset loading, memory, and retrieval remain. Its measured load time is 0.215 s; the first query takes 0.208 ms.

Keeping the index is the reason to consider this family. In a new-index decision, alternatives deserve their own evaluation. Nano’s BEIR-15 score is below the selected bge-small and LEAF references, which use their own document representations (Appendix A). BM25 and inference-free sparse retrieval also offer cheap queries. Constella is not a comprehensive small-model ranking.

3. Encoding savings and approximate-search cost

3.1 Where the encoding saving goes

Approximate nearest-neighbor search (**ANN**) explores a limited set of candidates rather than scoring every document. We measured all three encoders with Qdrant 1.19.1 on FiQA (57,638 documents) and a one-million-passage MS MARCO diagnostic. The latter retains every judged positive and samples the remaining passages, removing most full-corpus competitors. Its absolute scores and search penalties are not full-corpus MS MARCO results; it remains validation-only.

The registered sweep varies graph-search effort and compression. HNSW’s **ef** controls the search exploration budget. With compression, **oversampling** widens the candidates reconsidered using original vectors. Each compression configuration has its own collection; within a collection, documents and graph remain fixed for all query encoders.

At low search effort (**ef=16**) on the uncompressed million-passage collection, ANN loses 16.3% of Zero’s exact-search nDCG@10, compared with 7.0% for Nano and 4.4% for Stella. Raising search effort repairs much of this loss, but spends time. Earlier LightRetriever experiments showed a similar lookup-query penalty on FiQA; the size of the effect varied by dataset. Query-distribution effects on graph search are established in OOD-DiskANN and RoarGraph [OOD-DiskANN, RoarGraph]. These measurements test their practical consequence for this compatible family.

Table 2. On one shared binary collection, the fastest recorded settings within 1% of each encoder’s own exact score give:

Query path	ef / oversampling	ANN nDCG@10	Encode + search p50
Zero	512 / 2×	0.6116	1.11 ms
Nano	256 / 4×	0.6893	2.48 ms
Stella	128 / 1×	0.7226	21.3 ms

For example, suppose an application required a score of at least 0.68 on this diagnostic and a median encoding-plus-search budget of 3 ms. Among these three measured settings, Nano meets both. Zero falls below the relevance floor; Stella exceeds the time budget. If the budget were 2 ms, none of these rows would meet both requirements. These hypothetical thresholds illustrate the choice, not recommended production targets.

This exploratory restriction of the registered sweep keeps the physical index fixed. On these queries, Nano encoding is roughly 62 times slower than Zero; after search, the gap is 2.2-fold. Table 1 uses another query sample and implies a 51-fold encoding gap. On the uncompressed collection, Zero and Nano require 3.61 and 3.85 ms at the same tier-relative target, nearly consuming the advantage. FiQA’s best measured configurations leave a larger 3.8–4.3-fold gap.

The result separates compatibility from search efficiency. Shared document vectors do not establish that the original ANN settings remain sufficient. A target relative to each encoder’s own score isolates additional approximation loss; the absolute relevance requirement still determines which encoder is usable.

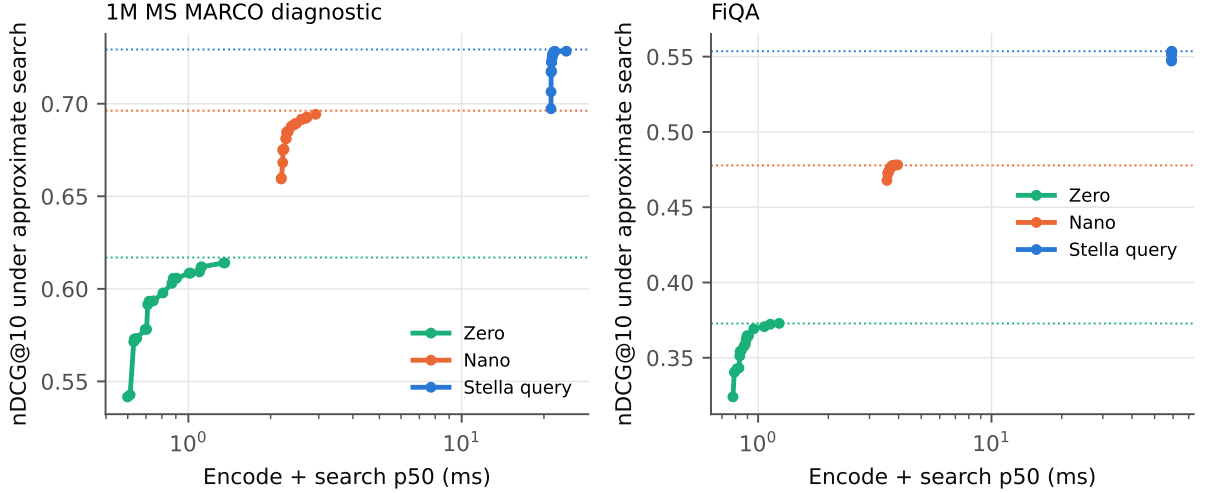


Figure 1: Measured quality/cost choices from the registered sweep. Curves can use different compression configurations; dotted lines show each encoder’s exact score. The shared-binary table above restricts all tiers to one collection. Time sums separately measured per-query encoding and search phases, excluding model loading and application dispatch. The quality levels are different.

3.2 Query precision deserves its own test

A binary document code stores coordinate signs. The tested engine’s default binary query encoding also keeps only signs, discarding query magnitudes [Qdrant 1.19.1]. We held document codes and query vectors fixed, removed the graph, and exhaustively compared $\text{sign}(\mathbf{q}) \cdot \text{sign}(\mathbf{d})$ with normalized $\mathbf{q} \cdot \text{the same } \text{sign}(\mathbf{d})$. The target remained each encoder’s original-vector exact top ten.

At a 40-candidate budget, preserving query magnitudes raises Zero’s original-neighbor coverage from 82.2% to 93.0% on FiQA and from 90.1% to 96.8% on the million-passage diagnostic. All three encoders improve. Zero starts with more misses and does not repair a larger fraction of them than the other tiers. The control shows that its cheap vectors still contain useful magnitude information; it does not measure a native latency or relevance gain. Query-aware binary scoring already has precedents [QA-Cos]. Appendix B preserves the controls and the weaker native replication.

Query precision is therefore a separate budget from encoder compute and document compression. A native comparison at matched quality and measured cost remains necessary to establish a faster production setting.

4. Lexical fusion and adaptive query computation

4.1 Lexical fusion is a practical alternative

Combining Zero with BM25 raises its BEIR-15 macro score from 0.4572 to 0.4933, or 87.9% of Stella’s score. Nano plus BM25 reaches 0.5110. These use distribution-based score fusion (**DBSF**), which normalizes and merges dense and lexical candidate scores, with 100 candidates per branch. Fusion adds a sparse index and retrieval path. On the million-passage diagnostic, a separate uncompressed dense-plus-sparse collection takes 4.71 ms including Zero encoding, versus 1.11 ms for the dense binary example. These are different configurations, not a pure fusion overhead measurement.

Fusion’s quality also varies by workload: HotpotQA Zero improves from 0.6127 to 0.7015, while MS MARCO Nano falls from 0.4063 to 0.3699. A later internal specialization attempt improved dense development results but failed its fused serving gate. Evaluate the route that will actually serve queries.

Earlier fusion research supplies a less obvious choice: **candidate depth changes the observed operator ordering**. On four development components, DBSF slightly exceeds fitted convex score fusion at ten candi-

dates but trails it at a thousand. Convex fusion rescales each branch’s scores and mixes them with the fixed development-fitted weight 0.8; DBSF uses the branch’s score distribution without fitting that weight. The lexical branch uses bm25s with Lucene settings. Other lexical implementations need their own comparison because their score distributions enter DBSF normalization.

Table 3. Fusion scores on the same development components, at different candidate depths.

Candidates per branch	Convex fusion	DBSF	DBSF – convex
10	0.5482	0.5517	+0.0035
50	0.5578	0.5558	−0.0020
100	0.5637	0.5574	−0.0063
1000	0.5727	0.5580	−0.0146

This development curve is descriptive. It does not establish equivalence at shallow depth or isolate a causal scoring mechanism. Our eventual DBSF@100 recommendation was an implementability decision after earlier evaluation; the original registered convex-fusion result remains in Appendix A. The lesson is to compare operators at the candidate budget and lexical implementation being deployed, including any self-document exclusion.

4.2 Routing and blending still need evidence

Compatible encoders permit a per-query choice, but an oracle using relevance labels is not a deployable policy. On 12 nonreserved BEIR datasets, a Zero/Nano oracle reaches 0.5473 versus always-Nano’s 0.5161. It matches always-Nano by sending the most beneficial 15% of each dataset’s queries to Nano. The simple policies we tested capture little of that opportunity.

Subwords per word, query length, and the pooled vector norm are weak selectors. Zero’s post-search score margin recovers about one quarter of the oracle gain over random routing on six datasets; escalated queries require another encoding and search. A cheap vector blend also disappoints: a development-selected 30% Zero contribution improves Nano locally but lowers its six-set score by 0.0051 (95% paired query interval −0.0092 to −0.0011).

These experiments distinguish available complementarity from realized benefit. Static encoder choices and lexical fusion have measured outcomes here; routing adds repeated searches, and the selected blend loses quality outside development. Better policies remain possible, consistent with work on retrieval-strategy selection and query performance prediction [Arabzadeh et al., QPP].

5. Teacher selection and construction cost

5.1 Teacher quality and table quality

An existing index fixes the teacher. Before building an index, a team can also choose which teacher its cheap query encoder will imitate. Earlier project screens reversed a teacher choice after evaluating its table. We followed that observation with a larger comparison: one **closed-form table recipe** across 26 checkpoints.

The probe fits token vectors using ridge regression, reconstructing teacher query vectors from token counts with a penalty against large changes from the starting rows. It uses 337,981 screened fit queries. The penalty is selected separately for each teacher on two development forums, then frozen before evaluation on six public BEIR datasets. Ten checkpoints were registered before observation; 16 were exploratory additions. One additional checkpoint failed the solver’s convergence requirement and is excluded. All share a BERT WordPiece vocabulary. Each table searches its own teacher’s document vectors: this is a teacher-plus-index selection study, not a way to swap unrelated teachers into Stella’s index.

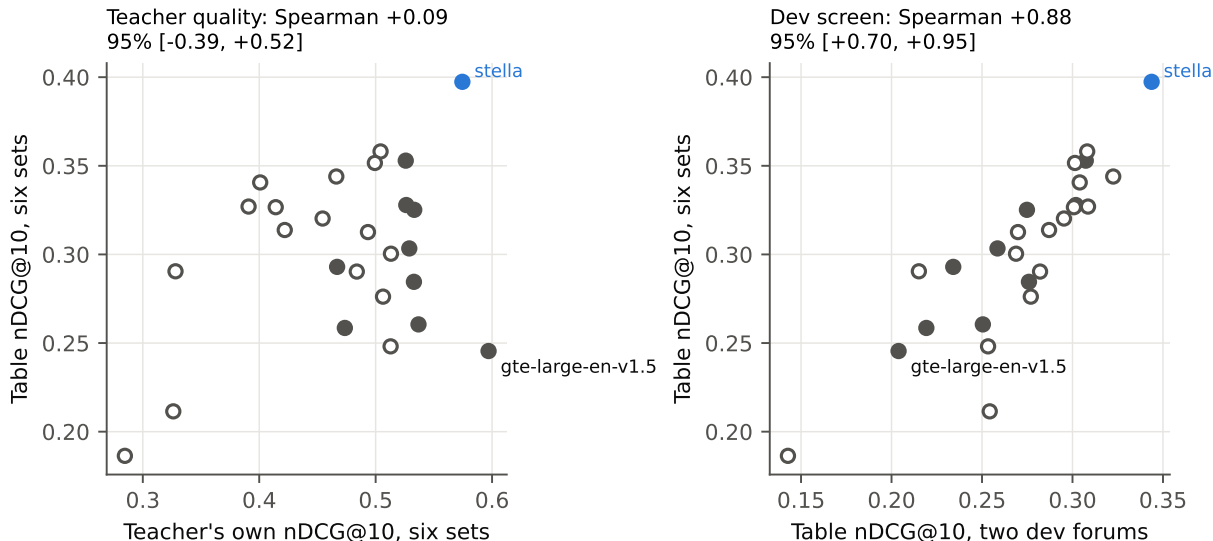


Figure 2: A direct table test ranks table retrieval much better than the teacher’s own score under this recipe. Filled points are the ten registered checkpoints; hollow points are 16 exploratory additions. Spearman correlation measures agreement between rankings. Intervals resample checkpoints, not training runs.

The strongest registered teacher, gte-large-en-v1.5, scores 0.5970 on the six datasets but produces the weakest registered table, at 0.2455. Stella’s teacher scores 0.5745 and its table 0.3974. Selecting the strongest teacher by that public score would therefore yield a substantially worse table. This is a hindsight comparison illustrating a selection failure, not a prospectively tested model-card policy.

Across all 26 checkpoints, teacher/table rank correlation is $+0.09$ (95% interval -0.39 to $+0.52$). The wide interval does not establish independence. Development-table/public-table correlation is $+0.88$ ($+0.70$ to $+0.95$), and $+0.90$ for the registered ten. The development test selects Stella, the best six-set average table. It is not best on every dataset; on the four sets without disclosed Stella overlap, ranking agreement drops to $+0.68$ and its selected table trails the best candidate. A representative development workload remains necessary.

Teacher strength is also not the same as ease of imitation. Tables fitted to two e5 checkpoints match teacher query vectors at mean cosine 0.90 and 0.89, versus 0.78 for Stella’s table, yet retrieve worse. Earlier Zero experiments and a later vocabulary-training screen likewise reduced training losses without improving ranking. These observations are consistent with prior work on stronger teachers producing weaker students [Cho and Hariharan 2019, PROD]. Our contribution is the consequential reversal and direct screen under this table recipe, rather than a general law of distillation.

The direct screen measures the representation whose retrieval is being selected. Teacher score, imitation fidelity, and optimization loss measure different outcomes. The probe predicts its own table recipe; it has not been validated as a selector for the separately trained Zero or Nano models.

5.2 Component build costs

The published models require no student training to use. Building another family requires texts, teacher-produced targets, fitting, export, and retrieval validation. Final Nano optimization took 57.3 hours on one A100 GPU for exactly 199,999,721 example presentations: about \$95 at the recorded historical rental rate. That is the optimization component, excluding target preparation, data generation, recipe search, failed runs, evaluation, and engineering.

Zero’s recorded retraining estimate was approximately 20 minutes on an RTX 3080 with prepared targets;

changing teachers was estimated to require another 8–12 hours of target encoding. Those are historical estimates rather than measured complete builds. These fitting components can run on a single GPU; they do not establish a turnkey reproduction budget or the minimum training needed.

6. Serving compatibility

An aligned checkpoint can still produce incompatible queries through the wrong serving path. In this project, missing Stella’s query prompt changes vector direction; padding to a fixed length corrupts a table path; a custom Nano bridge passes while native registration initially uses the wrong pooling. A candidate fp16 document export passes CPU parity but fails on CUDA, with minimum cosine 0.662 against its reference. We ship the fp32 path. These are implementation counterexamples, not claims that fp16 or exports generally fail.

The output contract also includes dtype. A mean-pooling wrapper promotes float32 embeddings to float64 through mask arithmetic, doubling vector bytes despite close numerical agreement. Naively narrowing before normalization can overflow float16 norms. Numerical agreement alone does not certify the output contract. Our serving qualification includes values, dtype, boundary lengths, and batch-size changes through the actual caller and device. Compatibility learning is established [BCT, FCT]; these failures concern preservation of the particular alignment being claimed.

7. Limits and conclusion

Constella’s budgets are measured properties of these trained artifacts. Zero and Nano use different training pools, including different FEVER exposure; their comparison does not isolate architecture. Stella also has disclosed exposure on some benchmark datasets. Appendix A separates original registered contrasts from later descriptive aggregates and preserves failed or unresolved tests.

Small internal specialization and patch experiments also encountered source-object leakage and judgment uncertainty. They did not establish an improved encoder and remain supporting evidence about evaluation limits in the companion research history.

The ANN and precision findings repeat across two workloads in one teacher space, on one laptop and sequential search. The million-passage subset removes competitors. Query intervals condition on fixed trained models; checkpoint intervals do not establish an independent-family law. The teacher screen uses one table recipe and is not validated for trained transformers. Concurrent switching, all-tier resident memory, server throughput, and a native precision remedy remain unmeasured.

The practical result is a choice of query budgets without replacing Stella’s document vectors. Making that choice useful requires testing the student’s retrieval, the candidate-search cost, the fused route, and the actual runtime. The next experiment with the clearest deployment value is a native query-precision comparison at matched recovery and relevance, including scoring cost. The current evidence justifies that experiment; it does not promise its outcome.

Appendix A. Evaluation and construction details

Serving configuration. Stella is pinned to revision `ffeb2b7e`, with its `s2p_query` prompt, fp32 encoding, normalized output, fp16 stored document vectors, and cosine similarity. Maximum sequence length is 512. Nano uses bge-small layers 12, 8, and 4, concatenated to 1152 dimensions before projection; batches mix 75% queries and 25% documents. Zero’s training pool includes FEVER-train and HotpotQA; Nano excludes FEVER. MS MARCO is validation-only in our distillation, although Nano’s pretrained base has prior exposure. Targets and permitted training sources are recorded in the companion build records.

Partitions and interpretation. The six-set suite is NFCorpus, SCIDOCS, SciFact, TREC-COVID, FiQA, and ArguAna. “Clean-four” means the first four have no disclosed Stella overlap; it is not a causal contamination estimate. Four originally reserved datasets were evaluated once: FEVER, DBpedia-entity, CQADup-Stack android, and english. Their published aggregates enter the broader descriptive evaluation and do not tune the paper’s diagnostics. NDO-3 weights DBpedia 0.5 and the two forums 0.25 each. Two other forums,

physics and programmers, select table penalties and routing/blend settings. Those development sources were repeatedly used during earlier model construction.

The external references below were selected project targets, not a comprehensive or state-of-the-art frontier. They use their own document spaces. Nano’s lower broad score is retained even though it passes some earlier registered contrasts.

Reference	Document representation	BEIR-15 macro nDCG@10	Encode p50
bge-small-en-v1.5	its own dense vectors	0.5171	2.57 ms
LEAF	arctic-embed-m dense vectors	0.5402	1.39 ms
BM25	lexical terms	0.4006	not reported here

Nano’s registered tests use a fixed sequence: each superiority conjunct is evaluated using a one-sided 2.5% lower bound at alpha 0.025. The original partitions and outcomes follow. Held-out intervals and other reported paired query intervals resample queries with trained models fixed. They do not include training-seed variation.

Original Nano contrast	Partition	nDCG@10 difference	Outcome
Nano — bge-small	clean-four	+0.017648; one-sided lower bound +0.003674	established
Nano — bge-small	all six	+0.027449	established
Nano — LEAF	all six	+0.016181	established
Nano — LEAF	clean-four	−0.001063	unresolved
Nano — bge-small	held-out NDO-3, dataset weighted	+0.0032 [−0.0069, +0.0134]	registered descriptive
Nano — bge-small	held-out NDO-3, query pooled	−0.0121 [−0.0219, −0.0024]	registered descriptive
Nano — LEAF	held-out NDO-3	−0.0389 [−0.0488, −0.0290]	registered descriptive

The original Zero tests used a Holm multiplicity rule across the confirmatory family. The table preserves their raw reporting intervals; these alone are not the decision rule.

Original Zero contrast, all six	nDCG@10 difference	Raw 95% interval	Registered outcome
Zero — LightRetriever dense	−0.0243	[−0.0405, −0.0086]	established below the 0.4583 bar
Zero — BM25	+0.0165	[+0.0017, +0.0311]	unresolved; p=.0149 fails Holm .0083
Zero + BM25 convex — OpenSearch	+0.0043	[−0.0063, +0.0151]	unresolved

The registered clean-four sensitivity is descriptive: the corresponding differences are −0.0443 [−0.0675, −0.0212], −0.0311 [−0.0517, −0.0109], and −0.0107 [−0.0262, +0.0043]. Zero therefore did not confirmatorily beat BM25. Unresolved superiority does not establish equivalence. The later DBSF recommendation does not replace the original confirmatory convex operator (weight 0.8, depth 1000).

The originally reserved four, copied from their published aggregate receipt:

System	FEVER	DBpedia	CQADup android	CQADup english
Nano	0.6231	0.4190	0.4774	0.4298
Zero	0.6978	0.3900	0.4431	0.3690
Stella	0.8207	0.4603	0.5275	0.5183
bge-small	0.8662	0.4003	0.4761	0.4558
LEAF	0.8671	0.4520	0.5260	0.4707
BM25	0.5036	0.3045	0.3952	0.3481
Zero + BM25	0.7559	0.4030	0.4629	0.4016
Nano + BM25	0.6879	0.4296	0.4750	0.4323

Teacher-screen limits. Checkpoint expansion follows the same selection order and recipe. The fit list was cleaned after an earlier version had 1.31% held-out-query overlap. Registered checkpoint exposure is documented; the exploratory expansion was not individually audited. A family-resampling sensitivity accompanies checkpoint intervals. Three vector-error diagnostics do not reliably explain teacher ranking. Direct table fitting takes roughly 4–7 minutes per registered configuration on an A100, inferred from output-file intervals and including query encoding except cached Stella targets; downloads, data assembly, and index construction are excluded.

Appendix B. Search and precision controls

The ANN sweep indexes every segment before timing, with HNSW `m=16`, `ef_construct=100`, and an indexing threshold of 1 KB rather than the 10,000 KB default. This avoids plain-scan segments in the benchmark and is not a deployment recommendation. Exact parity is checked against the original-vector NumPy reference. The positive-preserving million-passages sample is deterministic. Timing uses warmed, sequential queries and separately measured encoding/search phases; it does not include application dispatch, loading, or concurrent traffic.

The native binary follow-up alternates options on one fixed collection per workload, using 192 deterministically selected queries each. At primary `ef=64`, binary scoring with rescoring and $1\times$ oversampling loses 11.7/5.1/2.3 percentage points of original-neighbor recovery for Zero/Nano/Stella on FiQA. The million-passages replication loses 3.9/3.1/2.2 points; the extra Zero-versus-Nano interval includes zero. This weaker replication prevents a general static-model quantization claim. Native rescoring also changes cross-segment candidate merging, so it is not a rerank of an identical global pool.

The graph-free precision control uses exactly the same queries, vectors, and targets. It scores all fixed document sign codes, with global candidate budgets 10/40/100 and 40 primary. Expected coverage averages uniform inclusion at tied boundaries. The primary rows are:

Workload	Query path	Sign query	Full query magnitudes	Gain, percentage points
FiQA	Zero	0.8217	0.9297	+10.80
FiQA	Nano	0.9137	0.9771	+6.33
FiQA	Stella	0.9638	0.9922	+2.84
1M diagnostic	Zero	0.9010	0.9677	+6.67
1M diagnostic	Nano	0.9484	0.9833	+3.49
1M diagnostic	Stella	0.9685	0.9932	+2.47

The paired extra Zero/Nano gains are 4.46 points on FiQA (95% query interval 2.53–6.43) and 3.17 on the diagnostic (1.58–4.75). Mean cosine between full and sign-only queries is about 0.80 for every tier; the mechanism remains unresolved. Full query values are not native scalar8 encoding; global candidate counts

are not per-segment oversampling. All precision follow-ups are exploratory. Registered search and routing methods are distinguished from these controls in the evidence package.

Artifact availability

The [companion repository](#) contains inference examples, build configurations, methods, immutable result receipts, per-dataset tables, claim-to-evidence mappings, and the full milestone learning catalog. The released artifacts are [Constella Zero](#), [Constella Nano](#), and the [Stella document encoder](#). Model hashes and reference-vector checks identify the evaluated paths. The repository records construction but is not a standalone trainer for an arbitrary teacher.

References

- [QED] Yuxuan Wang and Hong Lyu. Query Encoder Distillation via Embedding Alignment is a Strong Baseline Method to Boost Dense Retriever Online Efficiency. SustaiNLP Workshop at ACL, 2023. [arXiv:2306.11550](#).
- [EmbedDistill] Seungyeon Kim, Ankit Singh Rawat, Manzil Zaheer, et al. EmbedDistill: A Geometric Knowledge Distillation for Information Retrieval. 2023. [arXiv:2301.12005](#).
- [LEAF] Robin Vujanic and Thomas Ruecksties. LEAF: Knowledge Distillation of Text Embedding Models with Teacher-Aligned Representations. 2025, revised 2026. [arXiv:2509.12539](#).
- [NanoVDR] Zhuchenyang Liu, Yao Zhang, and Yu Xiao. NanoVDR: Distilling a 2B Vision-Language Retriever into a 70M Text-Only Encoder for Visual Document Retrieval. 2026. [arXiv:2603.12824](#).
- [LightRetriever] Guangyuan Ma, Yongliang Ma, Xuanrui Gou, Zhenpeng Su, Ming Zhou, and Songlin Hu. LightRetriever: A LLM-based Text Retrieval Architecture with Extremely Faster Query Inference. ICLR, 2026. [arXiv:2505.12260](#).
- [pyNIFE] Stephan Tulkens. pyNIFE: Nearly Inference Free Embeddings in Python. Software, 2025. [Repository](#).
- [Model2Vec] MinishLab. Model2Vec: Fast State-of-the-Art Static Embeddings. Software. [Repository](#).
- [StaticEmb] Tom Aarsen. Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face, 15 January 2025. [Blog post](#).
- [Cho and Hariharan 2019] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation. ICCV, 2019. [Proceedings](#).
- [PROD] Zhenghao Lin, Yeyun Gong, Xiao Liu, et al. PROD: Progressive Distillation for Dense Retrieval. WWW, 2023. [arXiv:2209.13335](#).
- [Arabzadeh et al.] Negar Arabzadeh, Xinyi Yan, and Charles L. A. Clarke. Predicting Efficiency/Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. CIKM, 2021. [arXiv:2109.10739](#).
- [QPP] Guglielmo Faggioli, Thibault Formal, Stefano Marchesin, Stéphane Clinchant, Nicola Ferro, and Benjamin Piwowarski. Query Performance Prediction for Neural IR: Are We There Yet? 2023. [arXiv:2302.09947](#).
- [BCT] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards Backward-Compatible Representation Learning. CVPR, 2020. [Proceedings](#).
- [FCT] Vivek Ramanujan, Pavan Kumar Anasosalu Vasu, Ali Farhadi, Oncel Tuzel, and Hadi Pouransari. Forward Compatible Training for Large-Scale Embedding Retrieval Systems. CVPR, 2022. [arXiv:2112.02805](#).

- [OOD-DiskANN] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. 2022. [arXiv:2211.12850](#).
- [RoarGraph] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. PVLDB 17(11): 2735-2749, 2024. [Paper](#).
- [QA-Cos] Daehun Nyang. Beyond Hamming: Query-Aware Decoding of Binary Cosine Sketches. ICML 2026, PMLR 306: 94126-94143. [Proceedings](#).
- [Qdrant 1.19.1] Qdrant contributors. Binary query encoding and vector-index search implementation, v1.19.1. [Tagged source](#).
- [Stella] Stella-en-400M-v5. Model card for NovaSearch/stella_en_400M_v5. [Model page](#).
- [BEIR] BEIR contributors. BEIR retrieval benchmark, datasets, and evaluation software. [Repository](#).